

FedCare

Privacy-Preserving Federated Learning
for Heart-Disease Prediction

Phase 1: Code Explanation

A plain-English guide to every folder and file
in the Phase 1 codebase

1. The Big Picture - What Phase 1 Does

Phase 1 answers one simple question: "What accuracy can we get if we train a heart-disease prediction model in two extreme scenarios?"

Scenario A (Centralized): All 6 hospitals dump their patient data into one big pile and train one model together. This gives the BEST possible accuracy, but it means sharing private patient data - which is not allowed in real life.

Scenario B (Local Only): Each hospital trains its own model using ONLY its own patients. No data is shared, but each hospital has less data to learn from. This gives WORSE accuracy.

These two numbers (the upper bound and lower bound) are the benchmarks. In Phase 2, Federated Learning will try to match Scenario A's accuracy while keeping data private like Scenario B.

2. Project Folder Structure

```
Project Root/
|
|-- pyproject.toml          (Project settings)
|-- requirements.txt        (Libraries needed)
|-- conftest.py            (Test settings)
|
|-- fedcare/               (Our main code library)
|   |-- __init__.py
|   |-- task.py            (The brain - model + training)
|
|-- scripts/               (Helper scripts)
|   |-- prepare_data.py    (Converts Excel to CSVs)
|   |-- generate_dummy_data.py (Makes fake test data)
|
|-- data/heart/            (All patient data)
|   |-- Heart_Disease_...xlsx (Original Excel file)
|   |-- combined.csv        (All 12,000 patients)
|   |-- hospital_1.csv ... hospital_6.csv
|
|-- baseline_centralized.py (Scenario A script)
|-- baseline_local.py      (Scenario B script)
|
|-- tests/                 (Automated tests)
|   |-- test_baselines.py
```

3. Every File Explained in Simple English

pyproject.toml

Think of this as the project's ID card. It tells Python:

- The project is called 'fedcare' version 0.1.0
- It needs Python 3.11 or higher
- It depends on PyTorch, scikit-learn, pandas, etc.
- When running tests, only look inside the tests/ folder

Without this file, Python wouldn't know how to find our 'fedcare' code when we type 'from fedcare.task import Net'.

requirements.txt

A shopping list of all the Python libraries we need to install.

The version numbers are pinned (e.g., flwr==1.11.0) so that the code works the same way on any computer. If versions change, things might break.

conftest.py

A tiny configuration file for pytest (our test runner). It tells pytest: "Don't look inside the fedcare/ folder for tests - only look in tests/."

4. The fedcare/ Folder (Our Main Library)

This folder contains the core code that everything else depends on. Think of it as the engine of the car.

fedcare/__init__.py

An almost-empty file that tells Python "this folder is a Python package". Without it, we couldn't write 'from fedcare.task import Net'. It's like putting a nameplate on a door.

fedcare/task.py - THE MOST IMPORTANT FILE

This file contains 5 things. Everything else in the project uses these:

1) HeartDiseaseDataset (class)

A container that holds patient data (features + labels) in a format that PyTorch can understand. It takes a table of numbers and wraps them into tensors (PyTorch's version of arrays).

2) Net (class) - The Neural Network Model

A small neural network (called an MLP - Multi-Layer Perceptron) that takes 13 patient features as input and predicts whether they have heart disease (0 = no, 1 = yes).

Structure: 13 inputs -> 64 neurons -> 32 neurons -> 2 outputs

Total parameters: 3,042 (this is tiny - a phone could run it)

It uses ReLU activation (turns negative numbers to zero) and Dropout (randomly ignores 30% of neurons during training to prevent overfitting - like randomly quizzing students so they all study).

3) load_data() function

Reads a CSV file and prepares it for training. It does 3 things:

- Splits data into 80% training / 20% testing
- Scales all numbers to a similar range (StandardScaler) so the model doesn't get confused by large vs small values
- Wraps everything into DataLoaders (PyTorch's way of feeding data to the model in small batches of 32)

If you call load_data() with no argument, it loads ALL patients (combined.csv).

If you call load_data(partition_id=3), it loads only Hospital 3's patients.

4) train() function

Trains the model for one pass (epoch) through the data. In each step:

- Feed a batch of 32 patients through the model
- Check how wrong the predictions are (CrossEntropyLoss)
- Adjust the model's weights to reduce the error (Adam optimizer)
- Repeat for every batch

Returns the average loss (lower = better).

5) evaluate() function

Tests the model WITHOUT changing it. Feeds all test patients through the model and measures:

- Loss: How confident and correct the predictions are

FedCare - Phase 1 Code Explanation

- Accuracy: What percentage of predictions were correct
- AUC: How well the model separates sick from healthy patients (this is better than accuracy for imbalanced data)

5. The scripts/ Folder (Helper Tools)

scripts/prepare_data.py

This script reads your Excel file (Heart_Disease_Multi-Hospital_Research_Database.xlsx) and converts it into clean CSV files that the model can use.

What it does step by step:

1. Opens each hospital's sheet from the Excel file
2. Converts text columns to numbers:
 - sex: F=0, M=1
 - smoker: no=0, yes=1
 - diabetes_history: no=0, yes=1
 - family_history: no=0, yes=1
 - chest_pain_type: converted to 3 separate yes/no columns
(this is called 'one-hot encoding')
3. Drops the hospital_id column (not useful as a prediction feature)
4. Saves each hospital as hospital_1.csv ... hospital_6.csv
5. Combines all 6 into one big combined.csv

Result: 13 numeric features + 1 target column per file.

scripts/generate_dummy_data.py (Legacy)

This was used BEFORE you provided the real data. It created fake random patient records for testing the code. Now that we have real data, this file is no longer needed but is kept for reference.

6. The data/heart/ Folder (Patient Data)

Contains the raw and processed patient data:

Heart_Disease_Multi-Hospital_Research_Database.xlsx

Your original Excel file with 6 sheets (one per hospital) + a combined sheet.

hospital_1.csv through hospital_6.csv

Each file has 2,000 patients from one hospital. The data has already been preprocessed (text converted to numbers). Each hospital has a DIFFERENT mix of sick vs healthy patients - this is the Non-IID challenge:

Hospital	Patients	Disease Rate	Character
1	2,000	23.5%	Low disease
2	2,000	4.8%	Very rare disease
3	2,000	37.9%	High disease
4	2,000	20.7%	Low disease
5	2,000	46.3%	Near balanced

FedCare - Phase 1 Code Explanation

6	2,000	34.2%	Moderate disease
---	-------	-------	------------------

combined.csv

All 12,000 patients from all 6 hospitals merged into one file.

7. The Baseline Scripts (Our Experiments)

baseline_centralized.py (Scenario A: Share Everything)

Simulates the ideal but UNREALISTIC scenario:

"What if all 6 hospitals sent their data to one place?"

What it does:

1. Loads combined.csv (all 12,000 patients)
2. Creates the neural network model
3. Trains it for 20 rounds (epochs)
4. Prints accuracy and AUC every 5 epochs
5. Prints final results

Result: Accuracy = 80.8%, AUC = 0.848

This is the BEST we can do. Federated Learning aims to match this.

baseline_local.py (Scenario B: No Collaboration)

Simulates the privacy-respecting but LIMITED scenario:

"What if each hospital trains alone with only its own data?"

What it does:

1. Loops through hospitals 1 to 6
2. For each hospital: loads its CSV, creates a fresh model, trains it
3. Evaluates each hospital's model on its own test data
4. Prints a summary table comparing all 6 hospitals

Result: Average Accuracy = 80.7%, Average AUC = 0.812

Notice: Hospital 2 gets 95% accuracy but only 0.80 AUC. Why?

Because it has only 4.8% sick patients, so predicting 'healthy' every time gets 95% accuracy but is useless. AUC catches this trick.

8. The tests/ Folder (Quality Checks)

tests/test_baselines.py

Contains 10 automated checks that verify our code works correctly. Think of them as a safety checklist before a flight:

Model Tests:

- Can we create the model without errors?
- Does it produce the right output shape?

- Does it have exactly 3,042 parameters?

Dataset Tests:

- Can we create a dataset and check its length?
- Can we pull out one patient's data correctly?

Data Loading Tests:

- Can we load the combined CSV?
- Can we load one hospital's CSV?
- Are the data batches the right size?

Training Tests:

- Does training return a valid loss number?
- Does evaluation return loss, accuracy, and AUC?

9. How It All Connects (The Flow)

Here is the step-by-step flow of how the code runs:

```
Step 1: Run prepare_data.py
        Excel file --> hospital_1.csv ... hospital_6.csv + combined.csv

Step 2: Run baseline_centralized.py
        combined.csv --> load_data() --> train() x 20 --> evaluate()
        Result: Centralized accuracy = 80.8%

Step 3: Run baseline_local.py
        hospital_1.csv --> load_data(1) --> train() x 20 --> evaluate()
        hospital_2.csv --> load_data(2) --> train() x 20 --> evaluate()
        ... repeat for all 6 ...
        Result: Average local accuracy = 80.7%

Step 4: Run tests
        pytest checks all 10 safety checks --> All PASSED
```

10. What Comes Next (Phase 2)

In Phase 2, we add Federated Learning using the Flower framework.

Instead of sharing data (Scenario A) or working alone (Scenario B), the 6 hospitals will:

1. Each train the model on their OWN data (privacy preserved)
2. Send only the MODEL WEIGHTS (not patient data) to a central server
3. The server averages all the weights together (FedAvg algorithm)
4. Send the improved model back to each hospital
5. Repeat for several rounds

Goal: Match the centralized AUC of 0.848 without any hospital ever seeing another hospital's patients.

The key code addition will be a FlowerClient class that wraps our existing train() and evaluate() functions - so all the Phase 1 code we built is directly reused.